

Browser-Execution-Aware Detection of Command-and-Control Beacons Under Leakage-Free Evaluation

L. K. Wadduwage, Hansika Mahaadikara, Amila Senarathne
Department of Computer Systems Engineering
Sri Lanka Institute of Information Technology, Malabe, Sri Lanka
it22208408@my.sliit.lk, hansika.m@sliit.lk, amila.s@sliit.lk

Abstract—A browser extension or a compromised web page that beacons to a command-and-control (C2) server produces periodic HTTPS requests that are indistinguishable from ordinary browsing at the network-flow level. This paper presents a detector that observes outbound requests inside a managed browser through the DevTools Protocol, labels each request with browser-execution context that no network capture can supply – user activity, tab visibility, and request initiator – and fuses a calibrated gradient-boosted classifier with nine deterministic rules. The principal contribution is methodological. Evaluating by unseen malware family and by unseen C2 infrastructure, rather than by a random split, exposed two defects in the training corpus: one family’s positive windows were 95% drawn from a single communicating pair, and that same capture was a non-periodic dead-server retry storm labelled as a beacon. Both defects are diagnosed, a scope criterion independent of the timing features used for detection is defined, and a threshold-placement bug introduced by probability calibration is corrected. On the corrected corpus, under one reliability rule applied uniformly to both protocols, the model reaches 92.3% accuracy and 0.978 ROC-AUC on unseen C2 infrastructure and 83.8% accuracy and 0.927 ROC-AUC on an unseen malware family, class-balanced and leakage-free.

Index Terms—command-and-control detection, browser security, dataset leakage, probability calibration, behavioral anomaly detection

I. INTRODUCTION

Command-and-control beaconing is the step that turns a compromise into an operation: the implant contacts its server on a schedule, receives instructions, and returns results. Detection has traditionally been treated as a network problem and solved by looking for regularity – consistent inter-arrival times, consistent payload sizes, and a destination contacted at a cadence no human would produce [1]–[3]. Beaconing is a named and persistent adversary technique in attack frameworks [4], and malicious browser extensions remain an active and growing threat class [5].

When beaconing moves inside the browser – through a malicious extension, or script injected into a compromised page – the network view degrades. The beacon’s requests share the host’s own TLS session with genuine browsing to the same destination, so a beacon to a chat platform or document service is indistinguishable at the flow level from a user with that service open. Encryption removes the payload, and flow-level defences must fall back on side-channel features

to compensate [6]. What the network cannot see, the browser can: whether the user was interacting with the machine when a request fired, whether the tab was in the foreground, and whether an extension rather than a page initiated the request. This paper presents a browser-execution-aware detector built around that observation.

The principal contribution, however, is not the architecture but what happened when it was evaluated rigorously. A classifier trained on a real, published botnet capture produced strong held-out numbers under a random train/test split. Evaluating the same model with the malware family or specific attacker infrastructure held out entirely – the only protocol that reflects what a deployed detector actually faces – reduced those numbers sharply and forced a search for the cause. Two distinct corpus defects were found, rather than a modelling failure, together with a bug specific to the calibration step added to correct one of them. Concretely, this paper contributes:

- A non-invasive, browser-execution-aware collector on the Chrome DevTools Protocol producing a 29-feature per-host vector: 18 scale-free features read by the classifier, and 11 context or absolute-scale signals driving a deterministic heuristic layer, fused with fixed, measured weights (Section III).
- Diagnosis and correction of *pseudo-replication* – one family’s positive windows were 95% drawn from a single communicating pair – and of an *out-of-scope concept*, a non-periodic dead-C2 retry storm, removed by a scope criterion independent of the timing features used for detection (Section III-E, Section III-F).
- A threshold-placement bug specific to isotonic calibration, where a naive quantile rule is invalidated by calibration’s tied outputs, isolated by ablation (Section III-I).
- Leakage-free evaluation by unseen family and unseen C2 source under one reliability rule, alongside a five-classifier comparison showing *why* the deployed classifier was selected: an alternative wins the optimistic split and generalises to zero (Section IV).
- A measured structural result: with fixed fusion weights, confirmed detection is unreachable from the learned signal alone once browser context is unavailable, a property of the architecture, not an artefact of training (Sec-

tion IV-C).

II. LITERATURE REVIEW

A. Flow-Based Detection

Regularity statistics over inter-arrival times, payload-size consistency, and destination entropy underpin most beacon detectors. The CTU-13 capture set [1] remains a standard labelled corpus for this line of work; Gu *et al.* cluster communication and activity patterns independently of protocol [2], and Bilge *et al.* identify C2 servers from NetFlow records at scale [3]. More recent work decodes beaconing periodicity directly for threat hunting [7]. None of this literature has browser execution context available to it, by construction: it operates on network captures.

B. Browser-Mediated Threats

Malicious browser extensions are an active and growing category, and recent large-scale studies characterise their behaviour and prevalence [5]. Attributing traffic to a specific extension, tab, or script – rather than to a page’s own first-party traffic – is not a property that flow-level tooling can recover, which motivates instrumenting the browser directly.

C. Learning Under Scarce, Skewed Positives

Isolation Forest [8] targets settings where labelled anomalies are scarce relative to abundant normal traffic. Gradient-boosted trees such as XGBoost [9] are the supervised alternative once sufficient positives exist, but require explicit correction of their predicted probabilities: margin-based ensembles push scores away from 0 and 1, a distortion Niculescu-Mizil and Caruana correct with isotonic regression [10], which motivates the calibration step in Section III-I.

D. Evaluation Bias

Random or otherwise leakage-permissive splits over a corpus assembled from few sources overstate held-out performance. Sommer and Paxson argue that closed-world machine-learning evaluation transfers poorly to intrusion detection [11]; Pendlebury *et al.* formalise this for malware classification across space and time [12], and Arp *et al.* catalogue recurring pitfalls – sampling bias, spurious correlations, inappropriate measures – across three decades of security-ML papers [13]. The leave-one-family-out and leave-one-C2-source-out protocols used here apply that principle to a beacon corpus built from grouped, windowed captures; the pseudo-replication defect in Section III-E is a concrete instance of the sampling bias these authors describe.

E. Research Gap

The literature above establishes methods for flow-level regularity detection, documents browser extensions as a growing threat surface, and warns that evaluation protocol determines whether reported accuracy is credible. No prior work combines all three: a browser-execution-aware detector evaluated under a grouped, leakage-controlled protocol, with the corpus itself audited for pseudo-replication and label-concept scope. That combination is the gap this paper addresses.

TABLE I
THE 18 SCALE-FREE ML FEATURES, BY GROUP, WITH TRAINED IMPORTANCE

Group	Features	Imp.
Timing shape (8)	inter-arrival CV, Bowley skewness, normalized MAD, burstiness, lag-1 autocorrelation, spread ratio, clock-boundary share, normalized entropy	29.8%
Payload size (4)	mean, coefficient of variation, size-repeat ratio, upload/download ratio	19.6%
URL / method (5)	path entropy, unique-path ratio, POST ratio, normalized URI length, URI character entropy	18.8%
Request behaviour (1)	Referer-absent ratio	31.8%

III. METHODOLOGY

A. Non-Invasive Collection

Requests are captured through the Network domain of the DevTools Protocol rather than through request interception (Fig. 1). This is a performance decision: protocol events are fire-and-forget, so the browser never waits on the analysis process, whereas a global route handler pauses every outgoing request until it responds [14], [15], a real cost on any page issuing many concurrent requests. User interaction (click, keydown, scroll, touchstart) and tab visibility are recorded by an init script. Pointer movement is deliberately excluded, since it fires from passive cursor presence and inflates measured activity for exactly the two features that matter most. Requests from service workers and other non-page contexts are attributed to the background with the user marked inactive.

B. Feature Vector

Every 10 seconds, the rolling window of up to 50 requests per destination host – a fixed-size deque from which the oldest requests are dropped – is reduced to 29 features. Eighteen are scale-free (ratios, shares, normalised entropies) and are the only features the classifier reads (Table I); absolute byte counts or intervals were found to teach the model the scale of a 2011 capture rather than the property of beaconing (Section III-D). The most important trained feature, at 31.8% of total gain, is not a timing statistic: it is the share of requests carrying no `Referer` header, since a timer-fired request is not the result of a click. The remaining 11 features (inter-arrival mean and MAD, burst count, requests per hour, same-site ratio, script-initiator ratio, four context fields) drive the heuristic layer directly – absolute-scale or context features no offline network corpus can label.

C. Heuristic Layer and Fusion

Nine deterministic rules score, in turn: regular timing with a small payload; foreground requests firing during long user idle periods; background-tab traffic; extension-origin foreground beacon shape; low path entropy with regular timing and inactivity; script-initiated regularity; a high POST ratio with regular timing; a sustained high request rate while inactive;

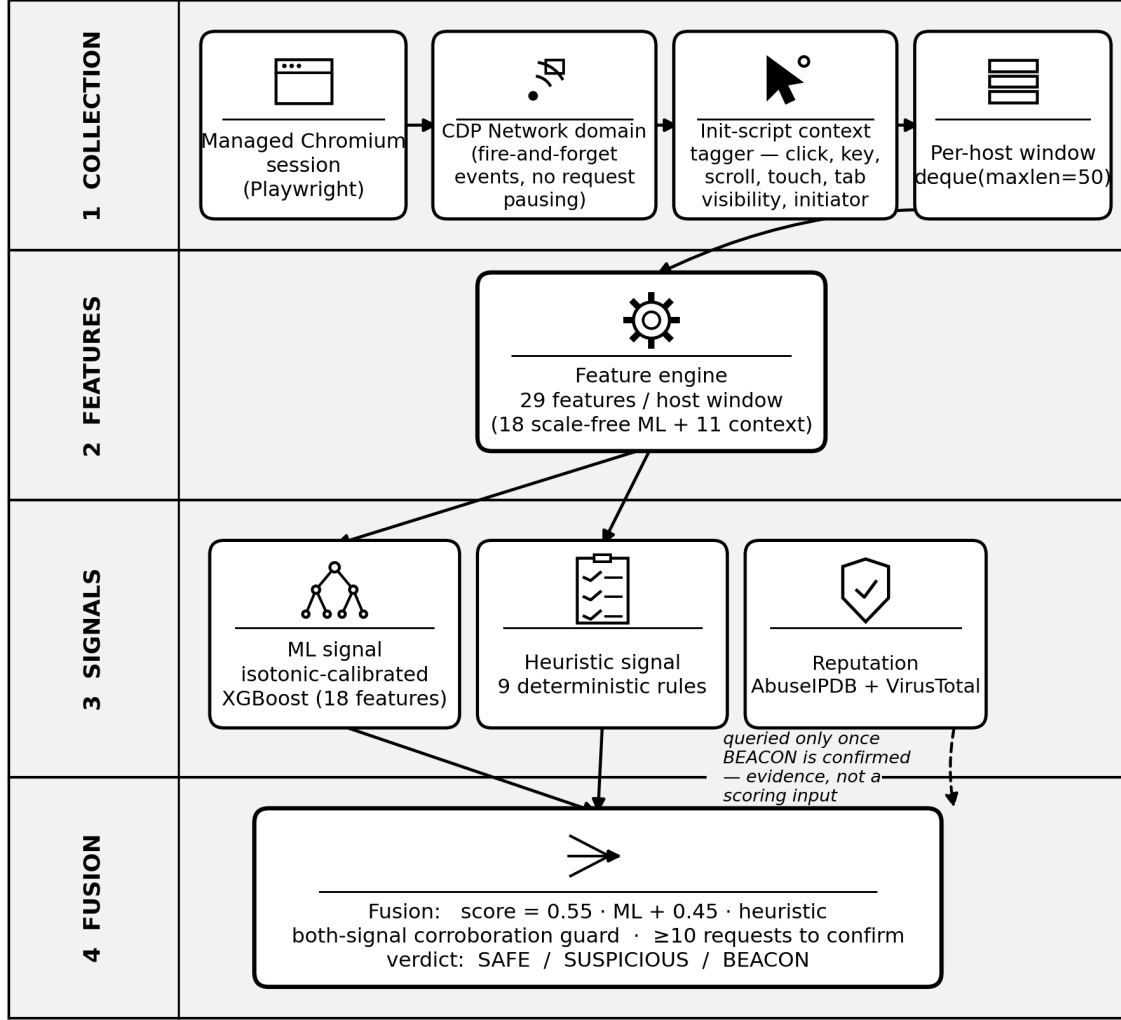


Fig. 1. The detection pipeline. Analysis runs every 10 seconds for each monitored destination host. Reputation is consulted only after a confirmed detection and is never an input to the score.

and, last and only multiplicatively, a same-site dampener that reduces – never raises – suspicion for traffic to the active page’s own site, so legitimate single-page-application background sync is not mistaken for a beacon.

The fused score is a fixed weighted sum, $\text{score} = 0.55 \cdot \text{ML} + 0.45 \cdot \text{heuristic}$, mapped to SAFE (< 0.30), SUSPICIOUS (< 0.52), or BEACON (≥ 0.52); Section IV-C reports how the weight was chosen. A both-signal corroboration guard withholds a BEACON verdict whenever either signal falls below a low floor, so a verdict never rests on one signal alone. A hard floor requires at least 10 observed requests before BEACON can be confirmed, and timing-dependent signals ramp in between 6 and 20 requests rather than switching on at a hard cutoff. Reputation services (AbuseIPDB, VirusTotal) are queried only after a BEACON verdict is confirmed, results

cached for 30 minutes as supporting evidence; reputation is never an input to the fused score.

D. Dataset Construction

The training corpus consists exclusively of real HTTP request records derived from public botnet captures, windowed into consecutive, non-overlapping blocks of real requests for each communicating pair. No row is synthetic. Before the corrections described below, the corpus contained 68,464 windows across six malware families, of which 6,890 were positive.

E. Pseudo-Replication

One family contributed 6,551 of those 6,890 positive windows (95.1%), but every one came from a single (src, dst) pair: one infected host, one C2 server, one continuous session

TABLE II
FEATURE MEDIANS, C2 WINDOWS ONLY: THE EXCLUDED CAPTURE IS
NOT PERIODIC

Feature	Excluded	Other C2	Benign
Inter-arrival CV↓	1.613	0.005	1.909
Clock-boundary share↑	0.082	1.000	0.095
Error-response ratio	1.000	0.043	0.267

cut into blocks. Counted by independent communicating pair rather than window, that family is 1 of 53, or 1.9%: a window count over a corpus built by slicing continuous sessions overstates the independent evidence available. This is a concrete instance of the sampling bias described in [12], [13], invisible to any evaluation that shuffles windows.

F. Out-of-Scope Concept

The same dominant capture is a dead C2: the server was unreachable and the malware was retrying into HTTP error responses. Its timing is retry backoff rather than a scheduled beacon (Table II), indistinguishable from ordinary browsing on the regularity feature the detector depends on most, and inverted relative to every other family on URI character entropy. Training on it forced the model to reconcile two different generative processes under a single label.

G. Scope Criterion and Cap

To avoid circularity with the timing features used for detection, scope is defined on channel outcome rather than on timing: a C2 window is in scope only if its error-response ratio is below 1.0, meaning the channel completed at least one successful exchange. A per-pair cap of 150 windows bounds the contribution of any single session. The resulting corpus contains 55,844 windows, of which 330 are positive, drawn from 50 independent C2 pairs across six families, and is used for training and for every evaluation result in Section IV.

Excluding the dead-C2 traffic from training did not remove the ability to detect it: scored against benign windows held out of training, it is caught 100% of the time at the deployed threshold, with ROC-AUC 0.995 against those negatives. Adding the error-response ratio as a nineteenth ML feature was considered and rejected: it separates the excluded capture almost perfectly (median 1.000) but sits on the *opposite* side of benign traffic for every other C2 family (0.043), so it would teach the model that the server is down rather than that the traffic is a beacon.

H. Classifier and Training

The classifier is XGBoost [9] with monotone constraints encoding directional domain priors – for example, the predicted probability may only decrease as the inter-arrival coefficient of variation rises. It is fitted with per-row sample weights that equalise the training contribution of each malware family and equalise real browsing against lab-background benign traffic: re-weighting rather than resampling, so no window is duplicated or invented.

I. Calibration and Threshold Placement

Motivated by the finding that boosted-tree probabilities are systematically distorted [10], the model is wrapped in isotonic calibration with grouped inner cross-validation, implemented with scikit-learn [16]. The decision threshold is placed at a target false-positive rate measured on benign traffic – the $(1 - \text{fpr})$ quantile of the benign score distribution – with the target FPR chosen by inner grouped cross-validation on training data only, mirroring how a deployed sensor is tuned against its own environment’s benign traffic, using no attack labels.

A threshold-placement bug was found and fixed here. The first implementation placed this threshold with `numpy.quantile`, but isotonic regression’s output is a step function with heavy ties, so that quantile routinely lands on a value shared by many benign windows, and $\text{score} \geq \text{threshold}$ then admits all of them at once. A fold with ROC-AUC 0.927, with ranking clearly intact, produced a precision inconsistent with any real operating point because the false-positive budget had been silently exceeded. The rule was replaced with a tie-aware ascending scan for the smallest threshold whose *achieved* FPR on held-out benign traffic meets the budget. Changing only this rule, leave-one-family-out F1 moves from 0.7635 to 0.8289 while ROC-AUC is unchanged at 0.9266, confirming the bug affected threshold placement and not ranking.

J. Evaluation Protocol

Test sets are class-balanced – benign windows are subsampled to the positive count, and five independent draws are averaged – so accuracy and precision are meaningful rather than a restatement of a sub-1% prevalence. Decision thresholds are always chosen by inner grouped cross-validation on training data only, never on the fold being scored. Two held-out protocols are reported: **LOPO** (leave-one-C2-source-out), holding out one communicating pair at a time, representing a known family reaching new infrastructure; and **LOFO** (leave-one-malware-family-out), holding out an entire family, representing one never seen in training.

One reliability rule applies uniformly to both: a fold contributes to a reported mean only if it contains at least ten positive test windows, since three positives cannot distinguish a working detector from a fortunate one. Of the 50 in-scope C2 pairs, 15 form a LOPO fold at all, and four clear the ten-positive bar; of six malware families, three clear it. All means in Section IV are taken over those folds only.

IV. RESULTS AND DISCUSSION

A. Headline Results

Table III and Fig. 2 report both protocols for the deployed model. Every reported mean exceeds the 0.75 target on every metric, but two qualifications matter.

First, applying the reliability rule to LOPO is not a cosmetic choice: averaging all 15 pair folds instead – including 11 that contain only three positive windows each, on which the model is trivially correct – would report 96.2% accuracy and 0.991 ROC-AUC, visibly higher on every metric. That figure is listed

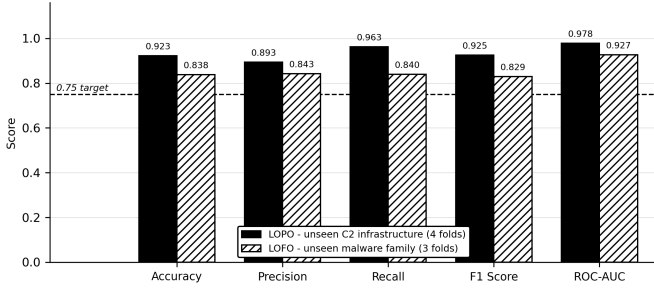


Fig. 2. Class-balanced held-out performance of the deployed model. Both protocols apply the same reliability rule: a fold enters a mean only if it contains at least ten positive test windows.

TABLE III

DEPLOYED MODEL, CLASS-BALANCED HELD-OUT EVALUATION. LOPO HOLDS OUT ONE C2 SOURCE; LOFO HOLDS OUT AN ENTIRE MALWARE FAMILY. MEANS COVER ONLY FOLDS WITH AT LEAST TEN POSITIVE TEST WINDOWS; THE FINAL ROW IS SHOWN FOR TRANSPARENCY AND IS NOT CLAIMED.

Protocol	Acc.	Prec.	Rec.	F1	AUC
LOPO (4 folds)	92.3%	89.3%	96.2%	92.5%	0.978
LOFO (3 folds)	83.8%	84.3%	84.0%	82.9%	0.927
ZeusV1 fold only	75.5%	86.9%	60.2%	71.1%	0.887
LOPO, all 15 folds	96.2%	94.8%	99.0%	96.6%	0.991

in Table III for transparency but is regarded as inflated by folds too small to carry information, and is not claimed.

Second, the mean is not the whole story: one held-out family, ZeusV1, reaches only 60.2% recall and 71.1% F1, below target, even though it does not pull the three-family mean below it. ZeusV1 is also the largest held-out family, contributing 226 of the 330 in-scope positive windows across three distinct C2 servers, so it is the best-supported single measurement in the LOFO set rather than a small-sample outlier, and is reported rather than averaged away.

B. Why the Gradient-Boosted Classifier Was Selected: A Five-Classifier Comparison

Logistic regression, Naive Bayes, a decision tree, Random Forest, and XGBoost were trained on the identical scoped corpus, with identical features and sample weights, and no per-model threshold tuning: a fixed 0.5 threshold at natural prevalence. This is a different, uncalibrated protocol from Table III, included only to justify the architecture choice. Table IV and Fig. 3 report the result. Random Forest wins the optimistic grouped split (F1 0.688, AUC 0.996) and then predicts *zero* positives on every held-out family: precision, recall, and F1 are exactly 0.000 across all six families, despite a still-informative ROC-AUC of 0.740 – its ranking survives the family shift, but no unseen-family window ever crosses 0.5. The split AUC of Naive Bayes (0.467, worse than chance) reflects its independence assumption being violated by strongly correlated timing features. XGBoost has both the best LOFO ROC-AUC (0.858) and the best LOFO F1 (0.267) of

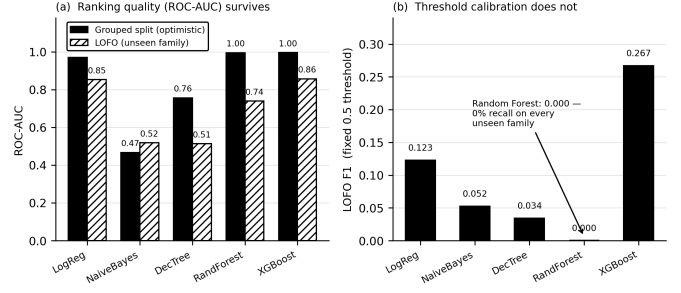


Fig. 3. Five classifiers on the identical scoped dataset and weighting. Random Forest wins the optimistic split, then fails to generalise: ranking quality partly survives an unseen family, but calibration to a fixed threshold does not.

TABLE IV

FIVE CLASSIFIERS, IDENTICAL SCOPED DATA, FIXED 0.5 THRESHOLD

Classifier	Split F1	LOFO F1	LOFO AUC
Logistic Regression	0.096	0.123	0.853
Naive Bayes	0.024	0.052	0.518
Decision Tree	0.640	0.034	0.514
Random Forest	0.688	0.000	0.740
XGBoost (deployed)	0.452	0.267	0.858

the five, the basis for its selection: cross-family ranking rather than in-distribution accuracy.

C. Fusion Weight and a Structural Limit

The ML weight was swept from 0.45 to 0.70 through the real fusion function on the full scoped corpus, under an assumed idle-background browser context. BEACON F1 peaks at weight 0.55 (0.9745, against 0.9324 at 0.45); beyond 0.55, precision falls faster than recall rises. This is the weight deployed in Section III-C.

Separately, under a context-blind condition where no browser-context fields are available – the most any network-only capture could ever supply – confirmed-BEACON recall measures **exactly 0.000 at every tested weight from 0.45 to 0.70**. This is not a training artefact: it follows from the both-signal corroboration guard withholding confirmation whenever the heuristic term sits at its floor, regardless of how the two weights are divided. SUSPICIOUS-tier detection is unaffected across the same range (F1 between 0.947 and 0.967), so nothing is silently missed; it surfaces at a lower confidence tier instead – the central operational claim of this paper: no re-weighting of the fusion substitutes for the browser-execution context the architecture is built to supply.

D. Discussion

These results together show that measured accuracy on this task is highly sensitive to evaluation protocol: the same corpus and classifier report either 96% or 84% depending on which folds are averaged. This sensitivity is not unique to gradient boosting – Random Forest’s optimistic-split win does not predict its cross-family behaviour at all – so protocol, not classifier choice alone, determines whether a reported number

holds up against new attacker infrastructure. The structural fusion result indicates that no algorithmic tuning substitutes for the browser-execution context the architecture supplies; this is a design constraint, not a shortcoming to optimise away later.

V. LIMITATIONS AND FUTURE WORK

- One held-out family, ZeusV1, falls below the 0.75 target on recall and F1, although the aggregate mean clears it (Section IV-A). Because it is the largest held-out family, this is a real limit of cross-family generalisation rather than a sampling artefact.
- Three of the six families are too small to evaluate meaningfully and are excluded from all reported means: Sogou and ZeusB26, with three held-out positive windows each, and Zeus78, which retains one window after the scope criterion is applied.
- Detection scope is active, periodic C2. A channel that never completes an exchange lies outside the trained concept, although it is separately shown to remain detectable (Section III-G).
- Confirmed BEACON detection architecturally requires corroborating browser context and is unreachable from the model alone without it (Section IV-C), which is a limitation for any deployment that cannot instrument the browser.
- The corpus predates modern jittered and malleable C2 frameworks. Extending coverage requires new labelled captures rather than further mining of the present ones.

Future work will extend the corpus to modern jittered and malleable C2 frameworks, which post-date the public captures used here and are known to defeat pure timing-regularity features, and will close the ZeusV1 gap (Section IV-A) by collecting additional labelled infrastructure for that family rather than by re-tuning the existing model. The context-blind result (Section IV-C) also motivates extending browser instrumentation to service workers, third-party iframes, and mobile DevTools Protocol variants, so the corroboration the fusion layer requires stays available across more deployments. Finally, connecting confirmed-BEACON verdicts to network-layer enforcement, such as an outbound firewall rule keyed to the flagged destination, would close the loop from detection to mitigation without a human in that step.

VI. CONCLUSION

Evaluating a browser-execution-aware C2 detector by unseen malware family and unseen infrastructure, rather than by a random split, did more than lower a number: it surfaced a pseudo-replicated training corpus, an out-of-scope traffic concept hiding inside the positive class, and a calibration-specific threshold bug, each diagnosed from first principles and corrected. Applying one reliability rule uniformly to both held-out protocols, the corrected model reaches 92.3% accuracy on unseen infrastructure and 83.8% on an unseen malware family, class-balanced and leakage-free; the higher figure a less careful aggregation would have produced is also reported and

explicitly not claimed. A five-architecture comparison shows the deployed classifier was chosen for surviving family shift, not for winning an optimistic split, since the alternative that wins the split generalises to nothing. The paper closes with a structural limitation measured directly: no fusion weight recovers confirmed detection from the model alone once browser context is unavailable, restating the paper’s central argument as an engineering fact.

ACKNOWLEDGMENT

The authors thank the members of research group R26-CS-003 for their collaboration on the shared WebSentinel platform, and the Stratosphere Laboratory for the public captures on which this work depends.

REFERENCES

- [1] S. Garcia, M. Grill, J. Stiborek, and A. Zunino, “An empirical comparison of botnet detection methods,” *Computers & Security*, vol. 45, pp. 100–123, 2014.
- [2] G. Gu, R. Perdisci, J. Zhang, and W. Lee, “BotMiner: Clustering analysis of network traffic for protocol- and structure-independent botnet detection,” in *Proceedings of the 17th USENIX Security Symposium*, 2008, pp. 139–154.
- [3] L. Bilge, D. Balzarotti, W. Robertson, E. Kirda, and C. Kruegel, “DISCLOSURE: Detecting botnet command and control servers through large-scale NetFlow analysis,” in *Proceedings of the 28th Annual Computer Security Applications Conference (ACSAC)*, 2012, pp. 129–138.
- [4] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, “MITRE ATT&CK: Design and philosophy,” MITRE Corporation, Tech. Rep., 2020, technical Report, revised March 2020.
- [5] S. Singh, G. Varshney, T. K. Singh, V. Mishra, and K. Verma, “A study on malicious browser extensions in 2025,” *arXiv preprint arXiv:2503.04292*, 2025.
- [6] B. Anderson and D. McGrew, “Identifying encrypted malware traffic with contextual flow data,” in *Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security (AISec)*, 2016, pp. 35–46.
- [7] A. Mahboubi, K. Luong, G. Jarrad, S. Camtepe, M. Bewong, and M. Bahutair, “Lurking in the shadows: Unsupervised decoding of beaconing communication for enhanced cyber threat hunting,” *Journal of Network and Computer Applications*, vol. 236, p. 104127, 2025.
- [8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008, pp. 413–422.
- [9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [10] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005, pp. 625–632.
- [11] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.
- [12] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, “TESSERACT: Eliminating experimental bias in malware classification across space and time,” in *Proceedings of the 28th USENIX Security Symposium*, 2019, pp. 729–746.
- [13] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, “Dos and don’ts of machine learning in computer security,” in *Proceedings of the 31st USENIX Security Symposium*, 2022, pp. 3971–3988.
- [14] Google Inc., “Chrome DevTools Protocol,” <https://chromedevtools.github.io/devtools-protocol/>, accessed 2026-09-03.
- [15] Microsoft Corporation, “Playwright for Python,” <https://playwright.dev/python/>, accessed 2026-09-03.
- [16] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.